Texnik topshiriq: SplitEasy — guruh xarajatlarini bo'lishish veb-ilovasi

Yo'nalish: Dasturiy injenering **Fan:** Dasturiy ta'minot loyihalash va testlash (amaliy loyiha) **Jamoa:** 3–4 talaba **Muddat:** 5 hafta (4 ta sprint + himoya) **Asosiy til:** Python 3.11+

---

1. Loyiha g'oyasi

SplitEasy — do'stlar, guruhdoshlar yoki yotoqxonadagi xonadoshlar birgalikda qilgan xarajatlarni yozib boradigan va "kim kimga qancha qarzdor" ekanini avtomatik hisoblab beradigan veb-ilova (Splitwise'ning soddalashtirilgan varianti).

Misol: 4 do'st sayohatga chiqdi. Ali mehmonxonaga 400 000 so'm to'ladi, Vali taksiga 100 000 so'm, Sardor ovqatga 300 000 so'm to'ladi. Ilova oxirida eng kam sondagi to'lov bilan hammani hisob-kitob qilib beradi: "Dilshod → Ali: 150 000 so'm".

Nega aynan shu g'oya?

Bu loyiha kichik, lekin **biznes mantig'i boy**. Shu sababli talabalar nazariyada o'rgangan narsalarni tabiiy ravishda amalda qo'llaydilar:

- xarajatni bo'lishning turli usullari → **Strategy** pattern va **Open/Closed** printsipi;
- xarajat qo'shilganda bildirishnoma yuborish → **Observer** pattern;
- ma'lumotlar bazasidan ajratilgan mantiq → **Repository** pattern va **Dependency Inversion**;
- sof hisoblash funksiyalari → **unit test** yozish uchun ideal;
- tashqi xizmatlar (email, valyuta kursi) → **mock** va **fake** ishlatish.

2. Loyihaning maqsadlari

2.1. Mahsulot maqsadi

Guruh a'zolari xarajatlarni tez kiritib, aniq va adolatli hisob-kitob natijasini ko'ra olishi.

2.2. O'quv maqsadlari (eng muhimi)

Loyiha oxirida har bir talaba quyidagilarni **amalda** bajargan bo'lishi kerak:

1. Kamida 3 ta SOLID printsipini real kodda qo'llash va buni asoslab berish.

2. Kamida 4 ta design patternni to‘g‘ri joyda qo‘llash.
3. `pytest` yordamida unit testlar yozish (mock/fixture bilan).
4. Kamida bitta funksiyani **TDD** (avval test, keyin kod) usulida yozish.
5. GitHub‘da jamoaviy ishlash: branch, Pull Request, code review, CI.
6. Qatlamli arxitektura (layered architecture) bo‘yicha loyiha tuzish.

Ma‘lumotlar bazasi bo‘yicha qo‘shimcha maqsadlar. Quyidagilar ham amalda bajarilishi kerak:

- Bazani normalizatsiya (3NF) asosida loyihalash va ERD chizish.
- Murakkab hisobot so‘rovlarini xom **SQL**da yozish (JOIN, GROUP BY, subquery, CTE, window function).
- **Indekslarni** ongli tanlash va ularning samarasini `EXPLAIN QUERY PLAN` hamda o‘lchov bilan isbotlash.
- Tranzaksiya, constraint va migratsiyalar bilan ishlash.

3. Foydalanuvchi rollari

Rol	Tavsif
Mehmon (Guest)	Ro‘yxatdan o‘tmagan foydalanuvchi. Faqat ro‘yxatdan o‘tish va kirish sahifalarini ko‘ra oladi.
Foydalanuvchi (User)	Tizimga kirgan, guruh yaratishi va guruhlarga qo‘shilishi mumkin.
Guruh egasi (Group Owner)	Guruhni yaratgan foydalanuvchi. A‘zo qo‘shadi/chiqaradi, guruhni o‘chiradi.
Guruh a‘zosi (Member)	Guruhda xarajat qo‘shadi, ko‘radi va o‘zining xarajatlarini tahrirlaydi.

4. Funksional talablar

Har bir talab **FR-raqam** bilan belgilangan. Prioritet: **M** = Must (majburiy), **S** = Should (kerak), **C** = Could (bonus).

4.1. Autentifikatsiya va foydalanuvchilar

ID	Talab	Prioritet
FR-1	Foydalanuvchi email, ism va parol bilan roʻyxatdan oʻta oladi. Email noyob boʻlishi shart.	M
FR-2	Parol kamida 8 belgi boʻlishi kerak va bazada **faqat xeshlangan** holda saqlanadi (bcrypt yoki argon2).	M
FR-3	Foydalanuvchi email va parol bilan kiradi va JWT token (yoki session cookie) oladi.	M
FR-4	Foydalanuvchi oʻz profilini (ism) tahrirlay oladi.	S

4.2. Guruhlar

ID	Talab	Prioritet
FR-5	Foydalanuvchi yangi guruh yarata oladi (nomi, valyutasi). Yaratuvchi avtomatik **egasi** boʻladi.	M
FR-6	Egasi guruhga email orqali boshqa roʻyxatdan oʻtgan foydalanuvchini aʼzo qilib qoʻsha oladi.	M
FR-7	Egasi aʼzoni guruhdan chiqara oladi, **faqat** aʼzoning balansi 0 boʻlsa.	M
FR-8	Foydalanuvchi oʻzi aʼzo boʻlgan guruhlar roʻyxatini koʻradi.	M
FR-9	Egasi guruhni oʻchira oladi (soft delete — bazadan jismonan oʻchirilmaydi).	S

4.3. Xarajatlar

ID	Talab	Prioritet
FR-10	Aʼzo guruhga xarajat qoʻsha oladi: tavsif, summa, kim toʻlagan, sana, ishtirokchilar.	M

ID	Talab	Prioritet
FR-11	Xarajat **to‘rt xil usulda** bo‘linishi mumkin (4.4-bo‘limga qarang).	M
FR-12	Xarajat qo‘shuvchi yoki guruh egasi xarajatni tahrirlay va o‘chira oladi.	M
FR-13	Guruhdagi xarajatlar ro‘yxati sana bo‘yicha tartiblangan holda, sahifalash (pagination) bilan ko‘rsatiladi.	M
FR-14	Xarajatlarni kategoriya (ovqat, transport, turar joy, boshqa) bo‘yicha filtrlash mumkin.	S
FR-15	Xarajatga izoh (comment) qoldirish mumkin.	C

4.4. Bo'lish usullari (Strategy pattern uchun asos)

1. **Teng bo‘lish (Equal):** summa ishtirokchilar soniga teng bo‘linadi.
2. **Aniq summalar (Exact):** har bir ishtirokchi uchun summa qo‘lda kiritiladi. Summalar yig‘indisi umumiy summaga teng bo‘lishi shart.
3. **Foiz bo‘yicha (Percentage):** har bir ishtirokchi uchun foiz kiritiladi. Foizlar yig‘indisi aynan 100 bo‘lishi shart.
4. **Ulushlar bo‘yicha (Shares):** har bir ishtirokchiga ulush beriladi (masalan, 2 kishi 1 ulush, 1 juftlik 2 ulush). Summa ulushlarga proporsional bo‘linadi.

Yaxlitlash qoidasi: pul summalari **butun so‘mda** (yoki eng kichik birlikda) saqlanadi, `float` ishlatish **taqiqlanadi** (`int` yoki `decimal.Decimal`). Agar bo‘lishda qoldiq qolsa (masalan, 100 / 3), qoldiq tiyinlari ketma-ket ishtirokchilarga 1 birlikdan taqsimlanadi, shunda yig‘indi har doim aniq umumiy summaga teng bo‘ladi.

4.5. Balans va hisob-kitob

ID	Talab	Prioritet
FR-16	Tizim guruhning har bir a‘zosi uchun **sof balansni** hisoblaydi (musbat — unga qarz, manfiy — u qarzdor).	M

ID	Talab	Prioritet
FR-17	Tizim "hisob-kitob rejasi"ni tuzadi: hammani nol balansga keltiradigan **minimal sondagi** o'tkazmalar ro'yxati.	M
FR-18	A'zo to'lovni "to'landi" deb belgilashi mumkin (settlement yozuvi). U balansga ta'sir qiladi.	M
FR-19	Foydalanuvchi barcha guruhlar bo'yicha umumiy holatini ko'radi: "Jami sizdan qarz: X, jami sizga qarz: Y".	S

Balansni hisoblash qoidasi:

- Xarajatni to'lagan odam: `+ (to'lagan summa)`
- Har bir ishtirokchi: `− (uning ulushi)`
- Settlement (A, B ga to'ladi): A balansi `+ summa`, B balansi `− summa`
- Guruhdagi barcha balanslar yig'indisi har doim **0** bo'lishi kerak (bu invariantni testlar tekshiradi).

Hisob-kitob rejasi algoritmi (greedy):

1. Balanslarni ikki ro'yxatga ajrating: kreditorlar (balans > 0) va debitorlar (balans < 0).
2. Eng katta qarzdor va eng katta kreditorni oling.
3. Ular orasida `min(|qarz|, kredit)` summalik o'tkazma yarating.
4. Ikkalasining balansini yangilang, nolga yetganini ro'yxatdan chiqaring.
5. Ro'yxatlar bo'shaguncha takrorlang.

4.6. Bildirishnomalar (Observer pattern uchun asos)

ID	Talab	Prioritet
FR-20	Yangi xarajat qo'shilganda, ishtirokchilarga bildirishnoma yaratiladi (ilova ichida).	M
FR-21	Settlement qayd etilganda, to'lov oluvchiga bildirishnoma yaratiladi.	M

ID	Talab	Prioritet
FR-22	Bildirishnomalar email orqali ham yuboriladi (ishlab chiqishda haqiqiy email o‘rniga konsolga/faylga yoziladigan "fake" yuboruvchi).	S
FR-23	Foydalanuvchi bildirishnomalarni "o‘qildi" deb belgilay oladi.	S

4.7. Hisobot va eksport

ID	Talab	Prioritet
FR-24	Guruh xarajatlarini **CSV** formatida eksport qilish.	S
FR-25	Eksport formatlari (CSV, JSON) osongina kengaytiriladigan bo‘lishi kerak.	S
FR-26	Guruh xarajatlarining kategoriya bo‘yicha jami yig‘indisi (oddiy diagramma yoki jadval).	C

5. Nofunksional talablar

ID	Talab
NFR-1	**Xavfsizlik:** parollar xeshlangan; SQL injection‘dan himoya (ORM yoki parametrlangan so‘rovlar); foydalanuvchi faqat o‘z guruhlarining ma’lumotlarini ko‘ra oladi (avtorizatsiya).
NFR-2	**Aniqlik:** pul hisob-kitoblarida `float` yo‘q.
NFR-3	**Kod sifati:** `ruff` (yoki `flake8`) va `black` xatosiz o‘tishi; barcha funksiyalarda type hint; muhim klasslarda docstring.
NFR-4	**Testlanuvchanlik:** biznes mantig‘i (domain/service qatlami) veb-freymvorksiz va bazasiz test qilinadigan bo‘lishi kerak.

ID	Talab
NFR-5	**Unumdorlik:** oddiy so'rovlar 500 ms ichida javob berishi (100 ta xarajatli guruhda). 100 000 xarajatli bazada ham asosiy ro'yxat so'rovlari indekslar yordamida 100 ms ichida bajarilishi kerak..
NFR-6	**Hujjatlashtirish:** `README.md` orqali loyihani 5 daqiqada ishga tushirish mumkin bo'lishi kerak.
NFR-7	**Qayta tiklanuvchanlik:** barcha bog'liqliklar `requirements.txt` yoki `pyproject.toml` da, versiyalari bilan.

6. Texnologiyalar steki

Qatlam	Texnologiya	Izoh
Veb-freymvork	**FastAPI**	Avtomatik Swagger hujjati; type hint bilan yaxshi ishlaydi. (Flask ham ruxsat etiladi.)
Ma'lumotlar bazasi	**SQLite** + **SQLAlchemy**	O'rnatish talab qilmaydi.
Frontend	**Jinja2 shablonlari** + oddiy HTML/CSS (yoki Bootstrap)	JavaScript minimal. Asosiy e'tibor backend arxitekturasida.
Validatsiya	**Pydantic**	FastAPI bilan birga keladi.
Testlash	**pytest**, **pytest-cov**, `unittest.mock`	
Sifat	**ruff**, **black**, (ixtiyoriy) **mypy**	
CI	**GitHub Actions**	
Parol xeshlash	**passlib[bcrypt]** yoki argon2	
Migratsiyalar	Alembic	Sxema o'zgarishlarini versiyalash; jadval va indekslar migratsiya orqali yaratiladi.

Qatlam	Texnologiya	Izoh
SQL tahlili	sqlite3 CLI yoki DB Browser for SQLite	So‘rovlarni qo‘lda sinash va EXPLAIN QUERY PLAN natijasini ko‘rish.

7. Arxitektura

Loyiha **to‘rt qatlamli** bo‘lishi kerak. Qatlamlar faqat **pastga** bog‘liq bo‘lishi mumkin.

```
[ API / Web qatlami ]      ← HTTP, formalar, shablonlar (FastAPI routerlar)
          ↓
[ Service qatlami ]        ← biznes mantig'i (GroupService,
ExpenseService...)
          ↓
[ Domain qatlami ]         ← sof Python: modellar, bo'lish strategiyalari,
balans hisoblash
          ↑
[ Repository qatlami ]     ← bazaga kirish (interfeys + SQLAlchemy
implementatsiyasi)
```

Muhim qoida: Domain qatlami hech qachon FastAPI, SQLAlchemy yoki boshqa tashqi kutubxonani import qilmaydi. Service qatlami repository‘larning **abstrakt interfeysiga** (`ABC` yoki `Protocol`) bog‘liq bo‘ladi, konkret SQLAlchemy klassiga emas.

Tavsiya etilgan papka tuzilmasi

```
spliteasy/
├── .github/
│   ├── workflows/ci.yml
│   ├── pull_request_template.md
│   └── ISSUE_TEMPLATE/
├── app/
│   ├── domain/
│   │   ├── models.py           # User, Group, Expense, Settlement
(dataclass)
│   │   ├── money.py            # pul bilan ishlash yordamchilari
│   │   ├── split_strategies.py  # Strategy pattern
│   │   ├── balance.py          # balans va hisob-kitob rejasi
│   │   └── events.py           # Observer uchun hodisalar
│   ├── services/
```

```
|   |   ├── auth_service.py
|   |   ├── group_service.py
|   |   ├── expense_service.py
|   |   └── notification_service.py
|   ├── repositories/
|   |   ├── interfaces.py        # abstrakt repository'lar
|   |   ├── sqlalchemy_repos.py  # haqiqiy implementatsiya
|   |   ├── in_memory_repos.py   # testlar uchun
|   |   └── report_queries.py    # xom SQL hisobot so'rovlari
|   ├── db/
|   |   ├── session.py           # ulanish, tranzaksiya, PRAGMA
foreign_keys=ON
|   |   └── orm_models.py        # SQLAlchemy jadvallari
|   ├── exporters/               # CSV, JSON eksport (Factory)
|   ├── api/                     # routerlar
|   ├── templates/
|   └── main.py
├── migrations/                  # Alembic migratsiyalari
├── scripts/
|   ├── seed.py                  # katta test ma'lumotlari generatori
|   └── benchmark.py             # indekssiz / indeks bilan o'lchash
├── docs/
|   ├── DESIGN.md
|   ├── DB.md                    # normalizatsiya, indekslar, benchmark
|   └── erd.png
├── tests/
|   ├── unit/
|   └── integration/
├── requirements.txt
├── pyproject.toml
└── README.md
```

8. Majburiy design patternlar

Har bir patternni qo'llash jamoa tomonidan `docs/DESIGN.md` faylida **asoslanishi** kerak: *qaysi muammoni hal qiladi, nega aynan shu yerda, patternsiz kod qanday bo'lardi*.

#	Pattern	Qayerda qo'llanadi	Talab
1	**Strategy**	Xarajatni bo'lish usullari (teng, aniq, foiz, ulush)	Umumiy `SplitStrategy` interfeysi va 4 ta implementatsiya. Yangi usul qo'shilganda mavjud kod o'zgarmasligi kerak.

#	Pattern	Qayerda qo'llanadi	Talab
2	**Repository**	Bazaga kirish	`UserRepository`, `GroupRepository`, `ExpenseRepository` interfeyslari. Ikki implementatsiya: SQLAlchemy va In-Memory.
3	**Observer** (yoki Event Bus)	Bildirishnomalar	Xarajat/settlement hodisasi yuzaga kelganda ro'yxatdan o'tgan kuzatuvchilar (in-app, email) chaqiriladi. `ExpenseService` bildirishnoma klasslarini bilmasligi kerak.
4	**Factory**	Strategiyani va eksportyorni yaratish	`SplitStrategyFactory.create("percentage")`, `ExporterFactory.create("csv")`.
5 (bonus)	**Adapter**	Tashqi valyuta kursi API'si	Tashqi xizmat javobini ichki formatga o'giruvchi adapter.
6 (bonus)	**Decorator**	Xizmat chaqiruvlarini loglash yoki keshlash	Masalan, `@log_call` yoki keshlovchi repository wrapper.

9. SOLID printsiplari bo'yicha talablar

Jamoa quyidagi jadvalni to'ldirib, `docs/DESIGN.md` ga qo'shishi shart: har bir printsip uchun **kamida bitta aniq misol** (fayl va klass nomi bilan).

Printsip	Loyihadagi kutilayotgan qo'llanilishi
S — Single Responsibility	`ExpenseService` faqat xarajat mantig'ini boshqaradi; bildirishnoma, eksport va autentifikatsiya alohida klasslarda.
O — Open/Closed	Yangi bo'lish usuli qo'shish uchun faqat yangi klass yoziladi va factory'ga ro'yxatdan o'tkaziladi; mavjud kod o'zgarmaydi.
L — Liskov Substitution	Har qanday `SplitStrategy` implementatsiyasi bir-biri o'rnini bosa oladi; In-Memory va SQLAlchemy repository'lari bir xil kontraktga amal qiladi (testlar ikkalasida ham o'tishi kerak).
I — Interface Segregation	Katta "hamma narsa qiladigan" interfeys o'rniga kichik, maqsadli interfeyslar (masalan, `ExpenseReader` va `ExpenseWriter`, yoki `Notifier`).

Printsip	Loyihadagi kutilayotgan qo'llanilishi
D — Dependency Inversion	Servislar repository va notifier'larni konstruktor orqali oladi (dependency injection), ularning o'zi yaratmaydi.

10. Ma'lumotlar modeli

User: `id`, `email` (unique), `name`, `password_hash`, `created_at`

Group: `id`, `name`, `currency`, `owner_id`, `is_deleted`, `created_at`

GroupMember: `group_id`, `user_id`, `joined_at`

Expense: `id`, `group_id`, `description`, `amount` (int), `paid_by` (user_id), `category`, `split_type`, `expense_date`, `created_by`, `created_at`

ExpenseShare: `expense_id`, `user_id`, `share_amount` (int) — bo'lish natijasi shu yerda saqlanadi

Settlement: `id`, `group_id`, `from_user_id`, `to_user_id`, `amount`, `created_at`

Notification: `id`, `user_id`, `message`, `is_read`, `created_at`

10.1. Bazani loyihalash

Yuqoridagi ro'yxat faqat boshlang'ich nuqta: sxemani jamoa **o'zi** loyihalaydi, asoslaydi va himoya qiladi.

- **ERD** (`docs/erd.png`): barcha jadvallar, bog'lanishlar va ularning turi (1:N, M:N) ko'rsatilgan. Foydalanuvchi va guruh orasidagi M:N bog'lanish `GroupMember` orqali hal qilinadi.
- **Normalizatsiya:** sxema kamida **3NF** da bo'lishi kerak. `docs/DB.md` da har bir jadval uchun funksional bog'liqliklar qisqa tahlil qilinadi.
- **Cheklovlar (constraints):** `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE` (masalan, `user.email` va `GroupMember` dagi `(group_id, user_id)` juftligi) va `CHECK` (masalan, `amount > 0`, `share_amount >= 0`, `from_user_id <> to_user_id`) bazaning o'zida e'lon qilinadi. Faqat Python kodidagi tekshiruv yetarli emas.
- **Foreign key'lar:** har bir FK uchun `ON DELETE` qoidasi (`RESTRICT`, `CASCADE` yoki `SET NULL`) tanlanadi va nega shunday ekani yoziladi. SQLite'da har bir ulanishda `PRAGMA foreign_keys = ON` bajariladi, aks holda FK ishlamaydi.

- **Ongli denormalizatsiya:** `ExpenseShare.share_amount` hosila ma'lumot (uni strategiyadan qayta hisoblash mumkin). Uni nega saqlash kerakligi (balansni tez hisoblash, yaxlitlash natijasi o'zgarmasligi) va bundan kelib chiqadigan xavf (yig'indi `amount` ga teng bo'lmay qolishi) `DB.md` da asoslanadi va testda tekshiriladi.
- **Soft delete:** `is_deleted` ustuni so'rovlar va `UNIQUE` cheklovlariga qanday ta'sir qilishi tahlil qilinadi.

10.2. Xom SQL hisobot so'rovlari

Hisobotlar ORM orqali emas, **xom SQL**da yoziladi. Ular `ReportRepository` interfeysi (ISP, DIP) ortida, `report_queries.py` da joylashadi; servislar faqat interfeysni biladi. Kamida **6 ta** so'rov majburiy, 1 va 8-so'rovlar majburiy.

#	Hisobot	Kerakli SQL vositasi	Bog'liq talab
1	Guruh a'zolarining sof balansi	`JOIN`, `SUM`, `GROUP BY`, `COALESCE`	FR-16
2	Kategoriya bo'yicha jami xarajat	`GROUP BY`, `ORDER BY`	FR-26
3	Oylik xarajat dinamikasi	`strftime`, `GROUP BY`	FR-26
4	Eng ko'p to'lagan 3 a'zo	`JOIN`, `ORDER BY`, `LIMIT`	—
5	Hali birorta xarajatda qatnashmagan a'zolar	`LEFT JOIN ... IS NULL` yoki `NOT EXISTS`	—
6	Guruh o'rtachasidan katta xarajatlar	skalyar subquery	—
7	Xarajatlarning o'sib boruvchi yig'indisi	window function: `SUM() OVER (ORDER BY ...)`	—
8	Foydalanuvchining barcha guruhlar bo'yicha umumiy holati	`WITH` (CTE), `UNION ALL`	FR-19

Qoidalar:

- Barcha so'rovlar **parametrlangan** (`:group_id` shaklida). Satrlarni qo'shib so'rov yasash (string concatenation) taqiqlanadi.
- Har bir so'rov ustida izoh bo'ladi: u nimani qaytaradi va qaysi jadvallarni bog'laydi.
- 1-so'rov natijasi Python'dagi domain balans hisobi bilan **aynan bir xil** bo'lishi kerak (integratsion test shuni tekshiradi).

- `DESIGN.md` da bir-ikki jumla: nega aynan hisobotlar uchun ORM o‘rniga xom SQL tanlandi.

10.3. Indekslash

Indeks o‘qishni tezlashtiradi, lekin yozishni sekinlashtiradi va joy egallaydi. Shuning uchun har bir indeks **o‘lchov bilan** asoslanadi.

1. Kamida **4 ta** indeks yaratiladi, ulardan kamida bittasi **composite** (ko‘p ustunli) bo‘lishi shart.
2. Har bir indeks uchun yoziladi: qaysi so‘rovni tezlashtiradi va ustunlar tartibi nega aynan shunday.
3. `scripts/seed.py` katta ma‘lumot yaratadi: kamida **100 000 xarajat**, 300 000 ulush, 1 000 foydalanuvchi, 200 guruh. Generator deterministik bo‘ladi (`random.seed`), shunda natijalar qaytariladi.
4. `scripts/benchmark.py` har bir asosiy so‘rovni indekssiz va indeks bilan kamida 5 martadan ishga tushirib, o‘rtacha vaqtni o‘lchaydi.
5. Har bir so‘rov uchun `EXPLAIN QUERY PLAN` natijasi saqlanadi: `SCAN` o‘rniga `SEARCH ... USING INDEX` chiqishi ko‘rsatiladi.
6. Kamida **bitta** ustun uchun indeks **qo‘shmaslik** qarori asoslanadi (masalan, `is_deleted` : qiymatlari kam xilma-xil, indeks deyarli foyda bermaydi).

Namuna ro‘yxat (jamoa o‘z so‘rovlariga qarab tasdiqlaydi yoki o‘zgartiradi):

Indeks	Qaysi so‘rovga kerak	Izoh
`user(email) UNIQUE`	Kirishda email bo‘yicha qidiruv	`UNIQUE` cheklovi o‘zi indeks yaratadi
`expense(group_id, expense_date DESC)`	Guruh xarajatlari sana tartibida, sahifalash (FR-13)	Avval tenglik ustuni (`group_id`), keyin tartiblash ustuni
`expense(group_id, category)`	Kategoriya bo‘yicha filtr (FR-14)	Composite: ikkala shart bitta indeksda
`expense_share(user_id)`	Foydalanuvchi qarzi (FR-19)	Birlamchi kalit `(expense_id, user_id)` faqat `expense_id` bo‘yicha qidiruvga yordam beradi

Indeks	Qaysi so'rovga kerak	Izoh
`group_member(user_id)`	"Mening guruhlarim" (FR-8)	Birlamchi kalit `(group_id, user_id)` `user_id` bo'yicha qidiruvga yordam bermaydi
`notification(user_id, is_read)`	O'qilmagan bildirishnomalar	Composite

`docs/DB.md` dagi benchmark jadvali shu shaklda to'ldiriladi:

So'rov	Indekssiz (ms)	Indeks bilan (ms)	Reja (oldin → keyin)
Guruh xarajatlari, 1-sahifa			`SCAN` → `SEARCH`
Kategoriya bo'yicha filtr			
Foydalanuvchi umumiy qarzi (8-so'rov)			

10.4. Tranzaksiyalar

Bir nechta yozuvni o'zgartiradigan har bir amal **bitta tranzaksiyada** bajariladi: yo hammasi saqlanadi, yo hech narsa.

- Xarajat va uning barcha `ExpenseShare` yozuvlari bitta tranzaksiyada yoziladi.
- Xarajatni tahrirlashda eski ulushlarni o'chirish va yangilarini yozish bitta tranzaksiyada bo'ladi.
- Settlement va unga bog'liq bildirishnoma yozuvi bitta tranzaksiyada saqlanadi.
- Tranzaksiya chegarasi **service qatlamida** belgilanadi (repository'da emas), chunki servis qaysi amallar bir butun ekanini biladi.
- `DB.md` da ACID harflaridan har biri loyihada qayerda ko'rinishini 1–2 jumla bilan tushuntiring.
- Bonus: **Unit of Work** pattern; ikki parallel sessiya bilan SQLite blokirovkasini ko'rsatuvchi kichik demo.

10.5. Migratsiyalar va test ma'lumotlari

- Sxema **Alembic** migratsiyalari orqali yaratiladi. Har bir o‘zgarish alohida migratsiya (`upgrade` va `downgrade` bilan), indekslar ham migratsiya orqali qo‘shiladi.
- Bo‘sh bazadan `alembic upgrade head` bilan to‘liq sxema yaratilishi `README.md` da yoziladi, CI esa shuni tekshiradi.
- Qo‘lda yozilgan `schema.sql` va Alembic natijasi bir xil sxema berishi kerak.

10.6. N+1 muammosi va sahifalash

- Xarajatlar ro‘yxatini ulushlari bilan chiqarishda **N+1** muammosini avval ataylab hosil qiling va o‘lchang, keyin `selectinload` / `joinedload` yoki bitta `JOIN` so‘rov bilan tuzating. "Oldin va keyin" so‘rovlar soni `DB.md` da ko‘rsatiladi.
- Test SQLAlchemy‘ning `before_cursor_execute` hodisasi yordamida so‘rovlar sonini sanaydi va 100 ta xarajat uchun 3 tadan oshmasligini tekshiradi.
- Ro‘yxat `LIMIT` / `OFFSET` bilan sahifalanadi (FR-13). Bonus: 100 000 qatorli bazada `OFFSET` sekinlashishini o‘lchab, **keyset pagination** bilan solishtiring.

10.7. `docs/DB.md` mazmuni

1. ERD va jadvallar tavsifi.
2. Normalizatsiya tahlili va ongli denormalizatsiya asoslanishi.
3. Constraint‘lar jadvali (nima, qayerda, nega).
4. Xom SQL so‘rovlari ro‘yxati va ularning vazifasi.
5. Indekslar, `EXPLAIN QUERY PLAN` natijalari va benchmark jadvali.
6. N+1: oldin va keyin.
7. Tranzaksiya qarorlari va ACID tushuntirishi.

11. API spetsifikatsiyasi

Barcha endpointlar (ro‘yxatdan o‘tish va kirishdan tashqari) autentifikatsiya talab qiladi.

Metod	Yo‘l	Tavsif	Muvaffaqiyat kodi
POST	`/auth/register`	Ro‘yxatdan o‘tish	201
POST	`/auth/login`	Kirish, token qaytaradi	200

Metod	Yo'l	Tavsif	Muvaffaqiyat kodi
GET	`/groups`	Mening guruhlarim	200
POST	`/groups`	Guruh yaratish	201
POST	`/groups/{id}/members`	A'zo qo'shish (faqat egasi)	201
DELETE	`/groups/{id}/members/{user_id}`	A'zoni chiqarish (faqat egasi)	204
GET	`/groups/{id}/expenses`	Xarajatlar ro'yxati (`?page=` , `?category=`)	200
POST	`/groups/{id}/expenses`	Xarajat qo'shish	201
PUT	`/groups/{id}/expenses/{eid}`	Xarajatni tahrirlash	200
DELETE	`/groups/{id}/expenses/{eid}`	Xarajatni o'chirish	204
GET	`/groups/{id}/balances`	A'zolar balanslari	200
GET	`/groups/{id}/settle-plan`	Hisob-kitob rejasi	200
POST	`/groups/{id}/settlements`	To'lovni qayd etish	201
GET	`/groups/{id}/export?format=csv`	Eksport	200
GET	`/notifications`	Mening bildirishnomalarim	200
PATCH	`/notifications/{id}/read`	O'qildi deb belgilash	200

Xato kodlari: `400` noto'g'ri ma'lumot (masalan, foizlar yig'indisi ≠ 100), `401` autentifikatsiya yo'q, `403` ruxsat yo'q (guruh a'zosi emas), `404` topilmadi, `409` ziddiyat (email band).

Namuna so'rov (`POST /groups/1/expenses` , foiz bo'yicha bo'lish):

```
{
  "description": "Kechki ovqat",
  "amount": 300000,
  "paid_by": 1,
  "category": "food",
  "split_type": "percentage",
  "participants": [
    {"user_id": 1, "value": 50},
    {"user_id": 2, "value": 30},
    {"user_id": 3, "value": 20}
  ]
}
```

12. Testlash talablari

12.1. Umumiy

- Barcha testlar `pytest` bilan yoziladi va `tests/` papkasida joylashadi.
- **Domain va service qatlami uchun kod qamrovi (coverage) kamida 80%**, `pytest --cov=app` orqali o'lchanadi.
- Testlar bir-biriga bog'liq bo'lmasligi va istalgan tartibda o'tishi kerak.
- Test nomlari tavsiflovchi bo'lsin, masalan: `test_percentage_split_raises_error_when_sum_is_not_100`.
- Har bir test **Arrange–Act–Assert** tuzilmasiga amal qiladi.

12.2. Majburiy test senariylari (minimum)

Bo'lish strategiyalari:

- Teng bo'lish: 300 / 3 = 100, 100 / 3 da yig'indi aynan 100 bo'ladi (qoldiq to'g'ri taqsimlanadi).
- Aniq summalar yig'indisi umumiy summaga teng bo'lmasa xato chiqadi.
- Foizlar yig'indisi 100 bo'lmasa xato chiqadi.
- Ulushlar proporsional bo'linadi; ulush 0 yoki manfiy bo'lsa xato.
- Bo'sh ishtirokchilar ro'yxati va manfiy summa xato beradi.

Balans va hisob-kitob:

- Barcha balanslar yig'indisi har doim 0.

- Oddiy ikki kishilik holat to‘g‘ri balans beradi.
- Hisob-kitob rejasini bajargandan keyin hamma balanslar 0 bo‘ladi.
- Hisob-kitob rejasidagi o‘tkazmalar soni `N − 1` dan oshmaydi (N — qarz/kreditga ega a‘zolar soni).
- Settlement balansga to‘g‘ri ta‘sir qiladi.

Servislar (mock/fake bilan):

- Guruh a‘zosi bo‘lmagan foydalanuvchi xarajat qo‘sha olmaydi.
- Faqat egasi a‘zo qo‘sha oladi.
- Balansi 0 bo‘lmagan a‘zoni chiqarib bo‘lmaydi.
- Xarajat qo‘shilganda `ExpenseAdded` hodisasi yuboriladi va kuzatuvchi chaqiriladi (`Mock` yordamida tekshiriladi).
- Ro‘yxatdan o‘tishda parol ochiq holda saqlanmaydi.

API (integratsion):

- `TestClient` bilan kamida 8 ta endpoint testi (muvaffaqiyat va xato holatlari).

Ma‘lumotlar bazasi (integratsion, in-memory SQLite):

- `CHECK` cheklovi: nol yoki manfiy summali xarajat bazaga yozilmaydi.
- `FOREIGN KEY` buzilishi: mavjud bo‘lmagan guruhga xarajat qo‘shib bo‘lmaydi.
- Tranzaksiya: xarajat va uning ulushlarini yozish paytida xato yuz bersa, hech narsa saqlanmaydi (rollback).
- Xom SQL hisobot so‘rovlari kichik, qo‘lda tuzilgan ma‘lumotlarda kutilgan natijani qaytaradi.
- Xom SQL balansi va Python domain balansi **bir xil** natija beradi (kesishgan tekshiruv).
- N+1: 100 ta xarajatli ro‘yxat so‘rovi 3 tadan ortiq SQL so‘rovi yubormaydi.

12.3. Majburiy amaliyotlar

- **TDD:** har bir talaba kamida **bitta** funksiyani avval test yozib, keyin kod yozish usulida bajaradi. Buni commit tarixida ko‘rsatish kerak (avval `test: ...` commit, keyin `feat: ...` commit).
- **Mock/Fake:** servis testlarida In-Memory repository va `unittest.mock.Mock` ishlatiladi; tashqi xizmatlar (email) haqiqiy chaqirilmaydi.
- **Parametrlangan testlar:** kamida 3 ta testda `@pytest.mark.parametrize` ishlatiladi.
- **Fixture:** umumiy tayyorgarlik uchun `conftest.py` da fixture‘lar.

13. GitHub bilan ishlash qoidalari

13.1. Repozitoriy

- Jamoa a'zolaridan biri **GitHub Organization** yoki shaxsiy hisobda repozitoriy yaratadi, qolganlarni *collaborator* sifatida qo'shadi. O'qituvchi ham qo'shiladi.
- `main` branch **himoyalangan** (branch protection): to'g'ridan-to'g'ri push taqiqlanadi; birlashtirish uchun kamida **1 ta approve** va **CI muvaffaqiyatli o'tishi** shart.

13.2. Branch strategiyasi

- Har bir vazifa uchun alohida branch: `feature/split-strategies`, `fix/balance-rounding`, `test/expense-service`, `docs/design-md`.
- Branch umri qisqa bo'lsin (1–3 kun), katta branchlardan qoching.

13.3. Commit xabarlari (Conventional Commits)

```
feat: add percentage split strategy
test: add tests for equal split rounding
fix: correct remainder distribution
refactor: extract notifier interface
docs: describe observer usage in DESIGN.md
```

Bitta commit — bitta mantiqiy o'zgarish.

13.4. Pull Request va Code Review

- Har bir PR **GitHub Issue**ga bog'lanadi (`Closes #12`).
- PR tavsifi shablon bo'yicha: nima o'zgardi, nega, qanday test qilindi.
- Har bir PR'ni **boshqa** jamoa a'zosi ko'rib chiqadi. Review sharhlari mazmunli bo'lishi kerak (shunchaki "LGTM" yetarli emas — kamida bitta konstruktiv fikr yoki savol).
- Har bir talaba kamida **5 ta** boshqa a'zoning PR'ini ko'rib chiqishi shart.
- PR hajmi 400 qatordan oshmasligi tavsiya etiladi.

13.5. Issues va Project Board

- Barcha vazifalar GitHub Issues'da yoziladi; `GitHub Projects` kanban doskasi (To Do / In Progress / Review / Done) yuritiladi.
- Issue'larda label'lar: `feature`, `bug`, `test`, `docs`, `must`, `should`, `could`.

13.6. CI (GitHub Actions)

`.github/workflows/ci.yml` har bir PR va `main` ga push'da quyidagilarni bajaradi:

1. Python o'rnatish va bog'liqliklarni o'rnatish.
2. `ruff check .` va `black --check .`
3. `pytest --cov=app --cov-fail-under=80`

CI qizil bo'lsa, PR birlashtirilmaydi.

14. Ish bosqichlari (sprintlar)

Sprint 0 — Tayyorgarlik (1-hafta boshi, 2–3 kun)

- Repozitoriy, branch protection, CI skeleti, papka tuzilmasi, `README.md`.
- Vazifalarni Issue'larga bo'lish, rollarni taqsimlash.
- `DESIGN.md` ning dastlabki varianti (arxitektura sxemasi).

Sprint 1 — Domain yadrosi (1-hafta)

- `money.py`, domain modellari.
- `SplitStrategy` interfeysi va 4 ta strategiya **(TDD usulida)**.
- `SplitStrategyFactory`.
- Balans hisoblash funksiyasi.
- **Natija:** bazasiz, veb'siz ishlaydigan va testlar bilan qoplangan domain qatlami.

Sprint 2 — Servislar va ma'lumotlar bazasi (2-hafta)

- Repository interfeyslari va In-Memory implementatsiyasi.
- `GroupService`, `ExpenseService`, `AuthService` (In-Memory repo bilan testlanadi).
- Hisob-kitob rejasi algoritmi.
- SQLAlchemy modellari va repository implementatsiyasi.
- **Natija:** servis qatlami to'liq testlangan; ma'lumotlar bazasi ulangan.
- ERD, `schema.sql` va Alembic'ning birinchi migratsiyasi; constraint'lar (PK, FK, UNIQUE, CHECK).
- Xarajat va uning ulushlarini bitta tranzaksiyada yozish.

Sprint 3 — API va Observer (3–4-hafta)

- FastAPI routerlar, autentifikatsiya, avtorizatsiya.
- Event/Observer mexanizmi va bildirishnomalar.
- Eksport (Factory bilan).
- Integratsion testlar.
- **Natija:** Swagger orqali to‘liq ishlaydigan API.
- Xom SQL hisobot so‘rovlari (`report_queries.py`).
- `seed.py` bilan katta ma’lumot yaratish, `EXPLAIN QUERY PLAN` , indekslar va `docs/DB.md` dagi benchmark jadvali.

Sprint 4 — Frontend, sayqal va himoya tayyorgarligi (5-hafta)

- Jinja2 shablonlari: kirish, guruhlar, xarajat qo‘shish formasi, balans va hisob-kitob sahifasi.
- Refaktoring, kod ko‘rigi, qamrov bo‘shliqlarini yopish.
- `DESIGN.md` ni yakunlash, demo tayyorlash.

15. Jamoa ichidagi rollar (tavsiya)

Rollar sprintlar bo‘yicha **almashib turishi** tavsiya etiladi, shunda hamma har bir qatlam bilan ishlaydi.

Rol	Mas’uliyat
Tech Lead	Arxitektura qarorlari, PR’larni yakuniy ko‘rib chiqish.
QA Lead	Test strategiyasi, coverage nazorati, CI.
Backend Developer(lar)	Servis va API qatlami.
Scrum/Docs Master	Project board, uchrashuvlar, hujjatlar.

(3 kishilik jamoada rollar birlashtiriladi. Muhimi — **har bir talaba** domain mantig’i, test va PR review bilan shug’ullanishi kerak.)

16. Topshiriladigan natijalar

1. **GitHub repozitoriy** — ishlaydigan kod, tarix toza va mazmunli.
2. **README.md** — loyiha tavsifi, o‘rnatish va ishga tushirish, testlarni ishlatish bo‘yicha yo‘riqnoma.
3. **docs/DESIGN.md** — arxitektura sxemasi, har bir pattern va SOLID printsipi uchun asoslash (fayl/klass ko‘rsatilgan holda).
4. **Testlar** — coverage hisoboti (≥ 80%).
5. **GitHub Project board** va Issues tarixi.
6. **Demo** — 10 daqiqalik jonli namoyish.
7. **Individual refleksiya** — har bir talaba 1 sahifalik hisobot: nimani o‘rgandi, qaysi qarori xato bo‘ldi, qaysi patternni eng foydali deb topdi.

Qo‘shimcha (ma’lumotlar bazasi):

- `docs/DB.md` va `docs/erd.png`: ERD, normalizatsiya tahlili, indekslar asoslanishi, benchmark jadvali.
- `migrations/` papkasi (Alembic) va `scripts/seed.py`.

17. Baholash mezonlari (100 ball)

Mezon	Ball
Funksional talablarning bajarilishi (barcha **Must** majburiy)	15
Arxitektura va qatlamlarning to‘g‘ri ajratilishi	10
Design patternlarni to‘g‘ri qo‘llash va asoslash	15
SOLID printsiplarini qo‘llash va asoslash	10
Unit va integratsion testlar (sifati va qamrovi)	15
Ma‘lumotlar bazasi: loyihalash, SQL, indekslar, tranzaksiyalar	15
GitHub jarayoni: PR, review, commit tarixi, CI	10
Hujjatlar (README, DESIGN.md, DB.md)	5
Demo va individual himoya	5

Individual baho: umumiy ball jamoa natijasi, lekin har bir talabaning hissasi GitHub statistikasi (commit, PR, review) va himoyadagi savollarga javoblar orqali alohida baholanadi. Himoyada har bir talabaga loyihaning istalgan qismi bo‘yicha savol berilishi mumkin.

18. Bonus vazifalar (qo'shimcha ball)

- **Adapter:** tashqi valyuta kursi API‘si orqali turli valyutadagi xarajatlarni bitta valyutaga o‘girish (testda tashqi API mock qilinadi).
- **Takrorlanuvchi xarajatlar** (har oy ijara haqi).
- **Property-based testlar** (`hypothesis` kutubxonasi): "har qanday xarajat va bo‘lish usulida ulushlar yig‘indisi umumiy summaga teng".
- **Docker** bilan ishga tushirish (`Dockerfile` va `docker-compose.yml`).
- **Loyihani onlayn joylashtirish** (deploy).

19. Talabalar uchun maslahatlar

- **Avval domain, keyin web.** Birinchi hafta davomida brauzer va baza haqida o'ylamang — sof Python klasslar va testlar yozing.
- **Pul uchun `float` ishlatmang.** `0.1 + 0.2 != 0.3` — bu moliyaviy dasturda xato beradi.
- **Kichik PR'lar** tez ko'rib chiqiladi va kamroq konflikt keltiradi.
- **Patternni majburan qo'llamang.** Har bir pattern muammoni hal qilishi kerak; "chiroyli ko'rinishi uchun" qo'shilgan pattern xato hisoblanadi. `DESIGN.md` da buni asoslab bering.
- **Konfliktdan qo'rqmang.** `main` dan tez-tez `git pull --rebase` qiling.
- **Test — bu hujjat.** Yaxshi test nomi kodning nima qilishini tushuntiradi.
- **Indeksni taxmin bilan emas, o'lchov bilan qo'shing.** Avval `EXPLAIN QUERY PLAN` ga qarang, indeks qo'shing, keyin yana o'lchang.
- **ORM har doim yaxshi SQL yozmaydi.** Murakkab hisobotni xom SQLda yozing va natijasini Python hisobi bilan solishtiring.
- **Kichik bazada hamma narsa tez ishlaydi.** Indeks samarasini ko'rish uchun kamida 100 000 qatorli ma'lumot yarating.

20. Foydali resurslar

- FastAPI hujjatlari: fastapi.tiangolo.com
- SQLAlchemy hujjatlari: docs.sqlalchemy.org
- pytest hujjatlari: docs.pytest.org
- Conventional Commits: conventionalcommits.org
- Refactoring Guru (patternlar): refactoring.guru/design-patterns